

Guide Live Preview - Visual Builder Contentstack

Le Problème à Résoudre

Quand tu édites du contenu dans Contentstack Visual Builder, il doit savoir **quel élément HTML correspond à quel champ** dans ton entrée CMS. Sans cette information, il ne peut pas afficher les zones cliquables/éditables.

La Solution : `data-cslp`

Contentstack utilise un attribut HTML spécial `data-cslp` (Content Stack Live Preview) pour faire ce lien.



Les 2 Fonctions Clés

1. `addEditableTags(entry)` - Côté SDK Contentstack

Entrée brute CMS → `addEditableTags()` → Entrée avec propriété `$`

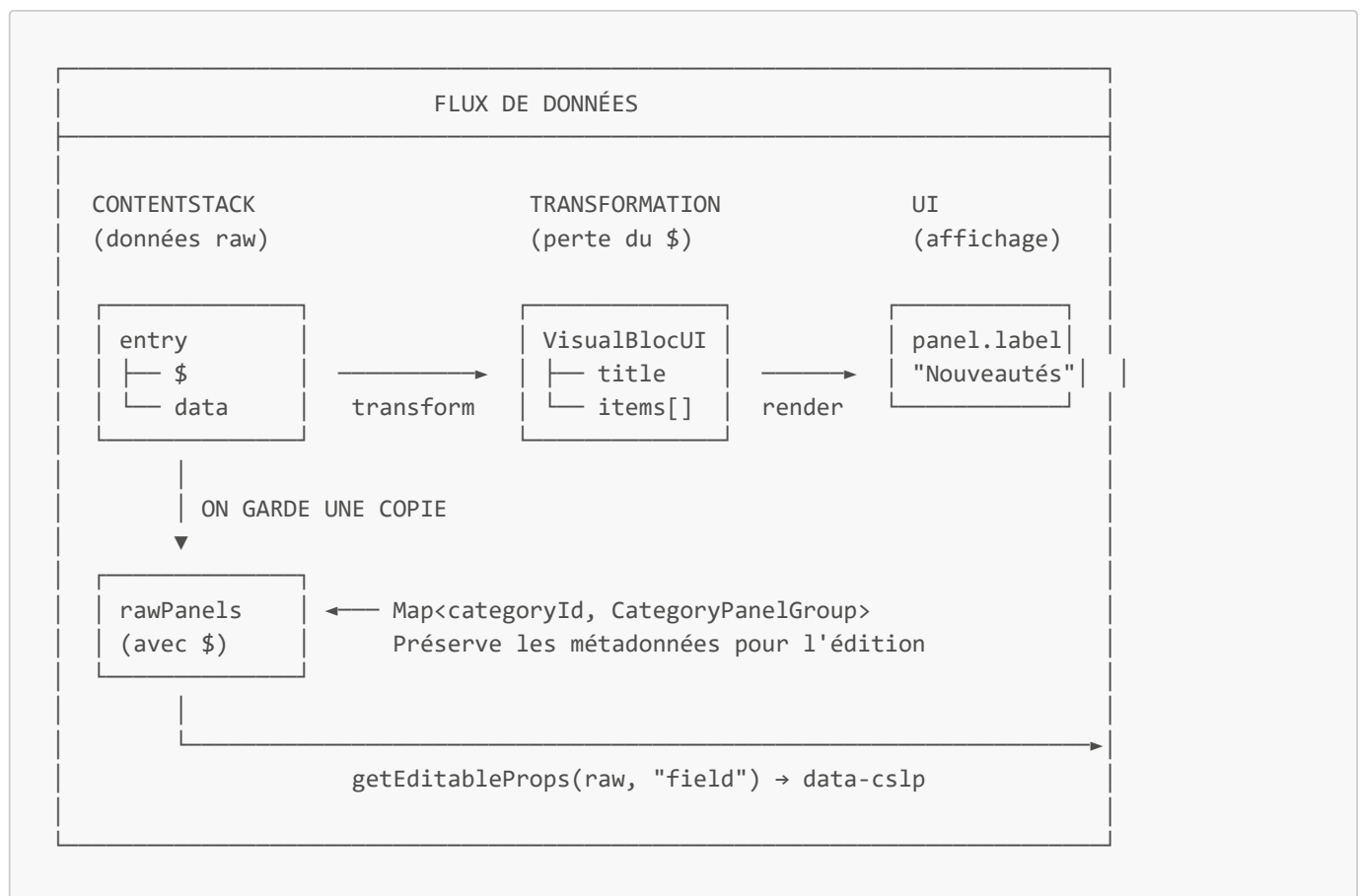
Cette fonction est appelée automatiquement par le SDK Live Preview. Elle ajoute une propriété `$` à chaque objet avec les chemins `data-cslp`.

2. `getEditableProps(obj, fieldName)` - Côté Composant

```
// Extrait le data-cslp pour un champ spécifique
getEditableProps(rawPanel?.visual_panel, "label")

// Retourne:
// { "data-cslp":
//   "menu_visual_bloc.categories_panels[0].category_panel_group.panels[0].visual_panel.label" }
// ou {} si pas de métadonnées
```

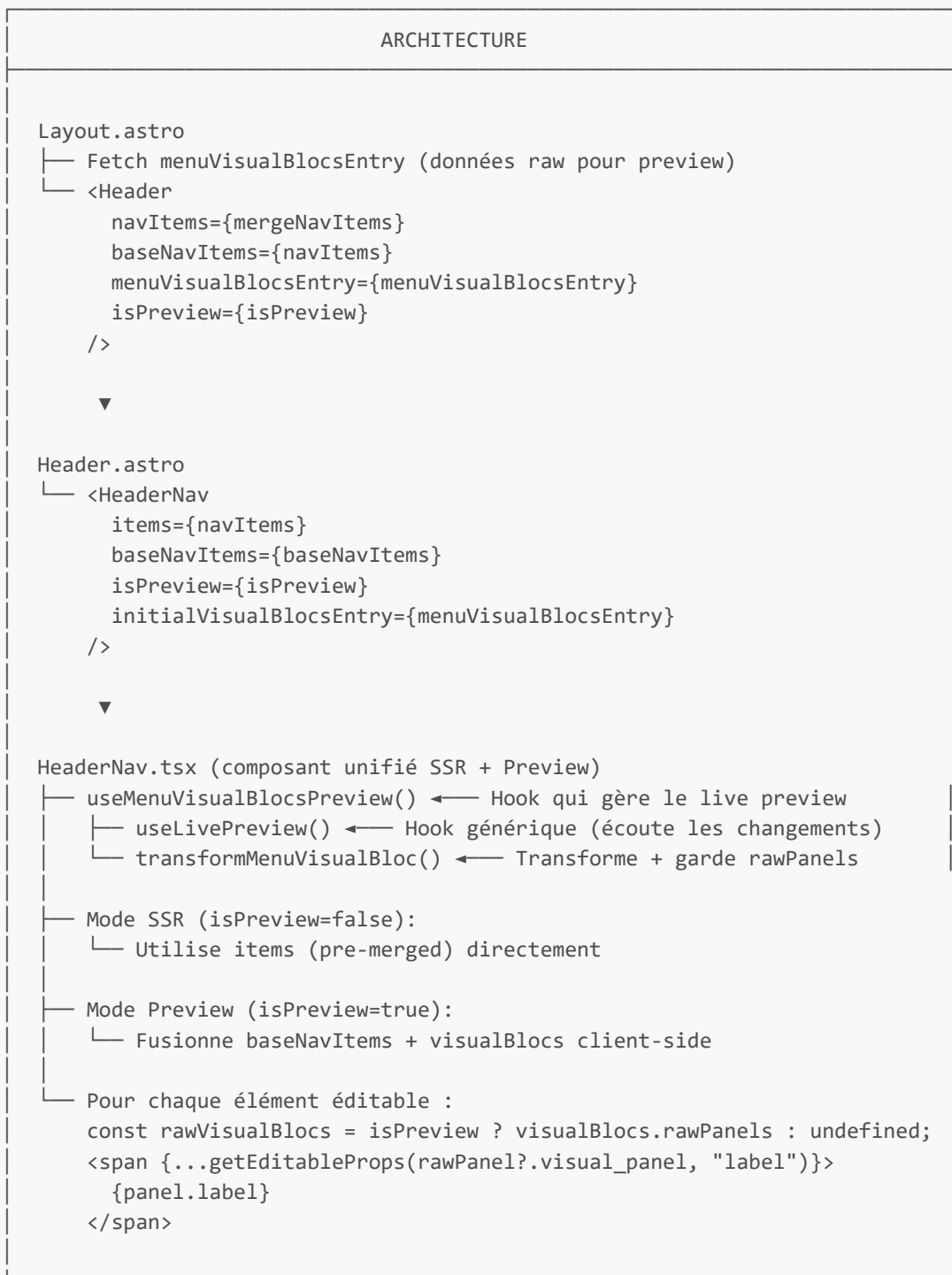
Pourquoi on a besoin des données "raw" ?



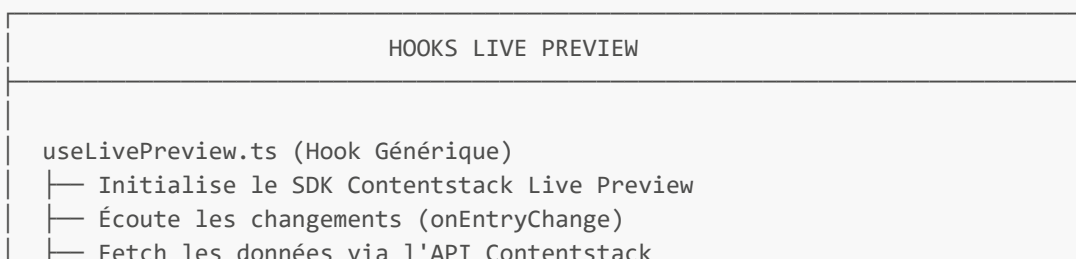
Le problème : Quand on transforme les données CMS en objets UI (`VisualBlocUI`, `VisualBlocItemUI`), on perd la propriété `$` avec les métadonnées.

La solution : On garde une copie des données brutes (`rawPanels`) pour pouvoir appeler `getEditableProps()` dessus.

Architecture des Fichiers (Pattern Unifié)



Hooks Live Preview



```

└─ Appelle addEditableTags() pour ajouter les métadonnées $
    ▲
    │ utilise
    │
    │ useMenuVisualBlocsPreview.ts (Hook Spécifique)
    │ └─ Appelle useLivePreview() avec contentType="menu_visual_bloc"
    │ └─ Transforme l'entry en VisualBlocsResult:
    │     └─ panels: Map<string, VisualBlocUI>
    │     └─ drills: Map<string, VisualBlocUI[]>
    │     └─ rawPanels: Map<string, CategoryPanelGroup> ← Pour édition!
    └─ Exporte mergeVisualBlocsIntoNavItems() pour fusionner

```

Syntaxe IIFE - Pourquoi et Comment

Le Problème

Dans JSX, tu ne peux pas déclarer de variables directement :

```

// ✗ INTERDIT - erreur de syntaxe
{items.map((item) => (
  const rawItem = getRaw(item); // ← ERREUR !
  <div>{item.name}</div>
))}

```

La Solution : IIFE (Immediately Invoked Function Expression)

```

// ☑ CORRECT - IIFE avec accolades + return
{(() => {
  const rawItem = getRaw(item); // ← OK, on est dans une fonction
  return (
    <div>{item.name}</div>
  );
})()}

```

Anatomie d'une IIFE

```

{(() => {
  const x = ...;
  return (
    <div>...</div>
  );
})()}

```

← Début IIFE : { pour JSX, (() => { pour la fonction
 ← Ici tu peux déclarer des variables
 ← Tu DOIS retourner du JSX
 ← Fin : })() appelle la fonction immédiatement

Comparaison des Syntaxes

```
// Arrow function avec return implicite (parenthèses)
items.map((item) => (
  <div>{item.name}</div>      // ← Pas de variables possibles
))

// Arrow function avec return explicite (accolades)
items.map((item) => {
  const x = doSomething();    // ← Variables OK
  return (
    <div>{x}</div>
  );
})

// IIFE dans JSX
{(() => {
  const x = doSomething();    // ← Variables OK
  return <div>{x}</div>;
})()}
```

Récapitulatif en 5 Points

#	Concept	Description
1	data-cslp	Attribut HTML qui lie un élément à un champ CMS
2	addEditableTags	Ajoute automatiquement la propriété \$ avec les métadonnées
3	getEditableProps	Extrait { "data-cslp": "..." } pour le JSX
4	rawPanels	Copie des données brutes pour garder les métadonnées \$
5	IIFE	Pattern {(() => { return ... })()} pour avoir des variables dans JSX

Pattern à Retenir

```
// Pour rendre un champ éditable en Live Preview :
<element {...getEditableProps(objetRawAvecDollar, "nomDuChamp")}>
  {valeurTransformée}
</element>
```

Exemple Complet (HeaderNav.tsx)

```
// Dans HeaderNav.tsx - composant unifié

export function HeaderNav({
  items: preMergedItems,
  baseNavItems,
  isPreview = false,
  initialVisualBlocsEntry,
  brand = "etam",
  // ... autres props
}: HeaderNavProps) {
```

```

// 1. Hook Live Preview pour visual blocs
const { visualBlocs, isLoading } = useMenuVisualBlocsPreview({
  brand,
  isPreview,
  initialData: initialVisualBlocsEntry,
});

// 2. Compute final nav items - SSR uses pre-merged, Preview merges client-side
const items = useMemo(() => {
  if (!isPreview && preMergedItems) {
    return preMergedItems;
  }
  if (baseNavItems) {
    return mergeVisualBlocsIntoNavItems(baseNavItems, visualBlocs);
  }
  return preMergedItems;
}, [isPreview, preMergedItems, baseNavItems, visualBlocs]);

// 3. Get raw visual blocs for editable props (only in preview mode)
const rawVisualBlocs = isPreview ? visualBlocs.rawPanels : undefined;

return (
  // ... dans le JSX, pour les visual panels :
  {(() => {
    const rawGroup = rawVisualBlocs?.get(visualPanelsItem?.key);
    return (
      <div className="header-nav__panel-inner">
        {/* Titre éditable */}
        <span
          className="text-body-strong-02"
          {...getEditableProps(rawGroup?.category_panel_group, "section_title")}
        >
          {visualPanelsItem.visualPanels.title}
        </span>

        {/* Items éditables */}
        {visualPanelsItem.visualPanels.items.map((panel, index) => {
          const rawPanel = rawGroup?.category_panel_group.panels[index];
          return (
            <a key={index} href={`/${panel.categorySlug}`}>
              {/* Image éditable */}
              <img
                src={panel.imageUrl}
                alt={panel.label}
                {...getEditableProps(rawPanel?.visual_panel, "image")}
              />
              {/* Label éditable */}
              <span {...getEditableProps(rawPanel?.visual_panel, "label")}>
                {panel.label}
              </span>
              {/* Title link éditable */}
              <span {...getEditableProps(rawPanel?.visual_panel, "title_link")}>
                {panel.title_link}
              </span>
            </a>
          );
        })}
      </div>
    );
  })}
);

```

```

    })()
  );
}

```

Checklist pour Ajouter le Live Preview (Pattern Unifié)

1. Côté Service (content.service.ts)

- ☐ Créer `getXxxEntry()` qui retourne les données raw (avec \$)

2. Côté Hook (useXxxPreview.ts)

- ☐ Créer un hook qui utilise `useLivePreview()`
- ☐ Transformer les données + garder les raw data dans un Map

3. Côté Composant (unifié SSR + Preview)

- ☐ Ajouter les props : `isPreview`, `initialXxxEntry`, `baseItems` (si fusion nécessaire)
- ☐ Appeler le hook preview dans le composant
- ☐ Calculer les items finaux via `useMemo()` (SSR vs Preview)
- ☐ Extraire `rawXxx = isPreview ? xxx.rawPanels : undefined`
- ☐ Sur chaque élément éditable : `{...getEditableProps(rawObject, "fieldName")}`

4. Côté Layout/Page (.astro)

- ☐ Fetch les données raw si `isPreview`
- ☐ Passer toutes les props au composant unifié

Structure JSON Contentstack - menu_visual_bloc

Voici la structure JSON renvoyée par Contentstack pour le content type `menu_visual_bloc` :

```

{
  "uid": "blt1234567890abcdef",
  "title": "Menu Visual Bloc - Etam",
  "brand": {
    "select_brand": "etam"
  },
  "categories_panels": [
    {
      "category_panel_group": {
        "parent_category_id": "vetements",
        "section_title": "Nos sélections",
        "visualpanel": true,
        "position_mobile": "bottom",
        "display_type_mobile": "landscape",
        "panels": [
          {
            "visual_panel": {
              "label": "Nouveautés",
              "title_link": "Voir tout",
              "category_id": "nouveautes",
              "image": {
                "uid": "blt_image_123",

```



```

    "_version": 5,
    "locale": "fr-fr",
    "ACL": {},
    "created_at": "2024-01-15T10:30:00.000Z",
    "updated_at": "2024-06-20T14:45:00.000Z",
    "created_by": "bltuser123",
    "updated_by": "bltuser456",
    "tags": [],
    "publish_details": {
      "environment": "production",
      "locale": "fr-fr",
      "time": "2024-06-20T15:00:00.000Z",
      "user": "bltuser456"
    }
  }
}

```

Après `addEditableTags()` - Avec métadonnées \$

Quand le SDK appelle `addEditableTags()`, chaque objet reçoit une propriété \$:

```

{
  "category_panel_group": {
    "parent_category_id": "vetements",
    "section_title": "Nos sélections",
    "panels": [
      {
        "visual_panel": {
          "label": "Nouveautés",
          "title_link": "Voir tout",
          "image": { "url": "..." },
          "$": {
            "label": {
              "data-cslp": "menu_visual_bloc.blt1234567890abcdef.fr-
fr.categories_panels.0.category_panel_group.panels.0.visual_panel.label"
            },
            "title_link": {
              "data-cslp": "menu_visual_bloc.blt1234567890abcdef.fr-
fr.categories_panels.0.category_panel_group.panels.0.visual_panel.title_link"
            },
            "image": {
              "data-cslp": "menu_visual_bloc.blt1234567890abcdef.fr-
fr.categories_panels.0.category_panel_group.panels.0.visual_panel.image"
            }
          }
        }
      }
    ]
  },
  "$": {
    "section_title": {
      "data-cslp": "menu_visual_bloc.blt1234567890abcdef.fr-
fr.categories_panels.0.category_panel_group.section_title"
    }
  }
}

```

Correspondance Champs → Édition

Champ Contentstack	Chemin dans l'objet	Fonction <code>getEditableProps()</code>
Section Title	<code>category_panel_group.section_title</code>	<code>getEditableProps(rawGroup?.category_panel_group, "section_title")</code>
Panel Label	<code>panels[i].visual_panel.label</code>	<code>getEditableProps(rawPanel?.visual_panel, "label")</code>
Panel Title Link	<code>panels[i].visual_panel.title_link</code>	<code>getEditableProps(rawPanel?.visual_panel, "title_link")</code>
Panel Image	<code>panels[i].visual_panel.image</code>	<code>getEditableProps(rawPanel?.visual_panel, "image")</code>

Fichiers de Référence

Fichier	Description
<code>src/lib/contentstack-sdk/useLivePreview.ts</code>	Hook générique Live Preview
<code>src/lib/contentstack-sdk/useMenuVisualBlocsPreview.ts</code>	Hook spécifique Menu Visual Bloc
<code>src/lib/contentstack-sdk/editable.ts</code>	Fonction <code>getEditableProps()</code>
<code>src/features/layout/Header/HeaderNav.tsx</code>	Exemple de composant unifié
<code>src/layouts/Layout.astro</code>	Exemple de passage des props